

## Kubernetes and Cloud Native Associate - KCNA

### Container Orchestration Networking

# CNI weave

Welcome to this article on the Weaveworks Weave CNI plugin. In this guide, we will delve into the inner workings of this CNI solution and explain how it compares and contrasts with traditional pod networking setups.

## Revisiting Pod Networking Concepts

Previously, we discussed custom CNI scripts integrated with kubelet. Often, to set up networking for a container, you might have executed:

```
./net-script.sh add <container> <namespace>  
ip -n <namespace> link set ...
```

In our earlier implementations, we replaced the custom script with the Weave plugin. Understanding the inner mechanics of the Weave solution will help you relate its operations to other networking solutions across Kubernetes.

## Traditional vs. Weave Networking Setup

In a manually configured networking setup, a routing table maps networks to hosts. When a packet moves from one pod to another, it exits the pod, enters the router,

and then is forwarded to the node hosting the destination pod. This approach works well for small networks, but in larger clusters—where hundreds of nodes and multiple pods exist per node—the routing table can become unwieldy.

Imagine a Kubernetes cluster as a company with various office sites (nodes). Consider the following analogy:

- **Traditional Approach:**

An employee in office 1 sends a package to office 3. An office messenger, using only general directions, makes the delivery. This system works in small companies but becomes increasingly complex as the number of sites grows.

- **Weave Approach:**

The company hires a dedicated shipping service. Each site has an assigned agent who maintains a complete directory of all routes. When a package is sent, the local agent repackages it with the correct destination, ensuring smooth delivery across multiple offices.



### Note

This analogy illustrates how Weave simplifies complex network topologies by delegating routing to dedicated agents on each node.

## How Weave Works

In a Kubernetes cluster using the Weave CNI plugin, an agent (or service) runs on every node. These agents share information regarding nodes, networks, and pods to maintain an accurate topology of the cluster. Here's how it functions:

- **Bridge Network:**

Weave creates a bridge on each node named "Weave" and assigns IP addresses to create an overlay network (the IP range can be confirmed through testing).

- **Multi-Bridge Connectivity:**

A single pod can be connected to multiple bridge networks (e.g., the Weave bridge and Docker's default bridge). The packet routing within the container directs traffic to the appropriate network interface.

- **Packet Encapsulation:**

When a pod sends a packet destined for a pod on another node, the local Weave agent intercepts it, encapsulates it with new source and destination addresses, and forwards it. On the destination node, the corresponding Weave agent decapsulates the packet and routes it correctly.

For example, to inspect the routing table from within a pod, use:

```
kubectl exec busybox -- ip route
default via 10.244.1.1 dev eth0
```

## Deploying Weave on a Kubernetes Cluster

After setting up your base Kubernetes cluster with properly networked nodes and control plane components, deploying Weave is straightforward. You can deploy Weave as a pod in your cluster using a single `kubectl apply` command. This command installs all necessary components including the essential Weave peers as a DaemonSet—ensuring one pod runs on every node.

To deploy Weave, execute:

```
kubectl apply -f "https://cloud.weave.works/k8s/net?k8s-version=$"
```

If your cluster was set up using kubeadm with the Weave plugin, you should see an output similar to:

```
kubectl apply -f "https://cloud.weave.works/k8s/net?k8s-version=$(kubectl version
serviceaccount/weave-net created
clusterrole.rbac.authorization.k8s.io/weave-net created
```

```
clusterrolebinding.rbac.authorization.k8s.io/weave-net created
role.rbac.authorization.k8s.io/weave-net created
rolebinding.rbac.authorization.k8s.io/weave-net created
daemonset.extensions/weave-net created
```

## Troubleshooting Weave

To inspect the logs and troubleshoot issues, list the pods in the `kube-system` namespace:

```
kubectl get pods -n kube-system
```

| NAME                    | READY | STATUS  | RESTARTS | AGE |
|-------------------------|-------|---------|----------|-----|
| corddns-78fcd6894-99khw | 1/1   | Running | 0        | 19m |
| corddns-78fcd6894-p7dpj | 1/1   | Running | 0        | 19m |
| etcd-master             | 1/1   | Running | 0        | 18m |
| kube-apiserver-master   | 1/1   | Running | 0        | 18m |
| kube-scheduler-master   | 1/1   | Running | 0        | 17m |
| weave-net-5gcm8         | 2/2   | Running | 1        | 19m |
| weave-net-fr9n9         | 2/2   | Running | 1        | 19m |
| weave-net-mc6sz         | 2/2   | Running | 1        | 19m |
| weave-net-tbzbz         | 2/2   | Running | 1        | 19m |

Use the `kubectl logs` command to review the logs of specific pods if issues arise.



### Warning

Ensure that your cluster's network policies and firewall rules allow communication between Weave agents, or connectivity issues may occur.

## In Summary

The Weave CNI plugin simplifies complex networking in large Kubernetes clusters by using dedicated agents to manage packet routing. It provides a scalable alternative to manually managed routing tables and makes inter-node communication more efficient.

For more detailed information on Kubernetes networking, you might find these resources helpful:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Weaveworks Documentation](#)

This concludes our discussion on the Weave CNI plugin. In future articles, we will explore IP address management and the methods used to assign IP addresses to pods and containers.



### Watch Video

Watch video content



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ▾